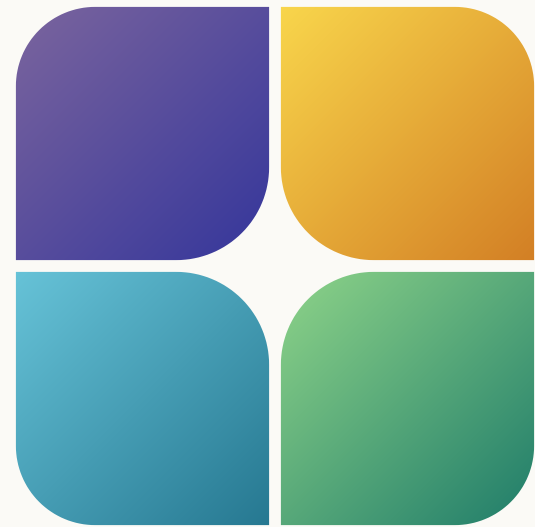


DINGG AI · NATURALS × HACKATHON WORKSHOP · 2026



# APIs & AI Toolkits

## *for beauty tech*

LLMs · Vision APIs · Agentic Coding · Integration patterns that ship.

**Santosh Patidar** Co-founder, dingg.ai



## AGENDA

# What we'll cover today

*40 minutes · three parts*

---

### PART 1

## The map

The beauty-tech AI stack, what's already shipping, four integration patterns, and where vision APIs fit in 2026.

---

### PART 2

## Agentic Coding

How to actually build with AI agents — vocabulary, SPEC-first, system design, testing, stack choice, the loop.

---

### PART 3

## The future

Ten predictions for beauty tech, three lenses to evaluate any idea, and your next step.

• THE INSIGHT

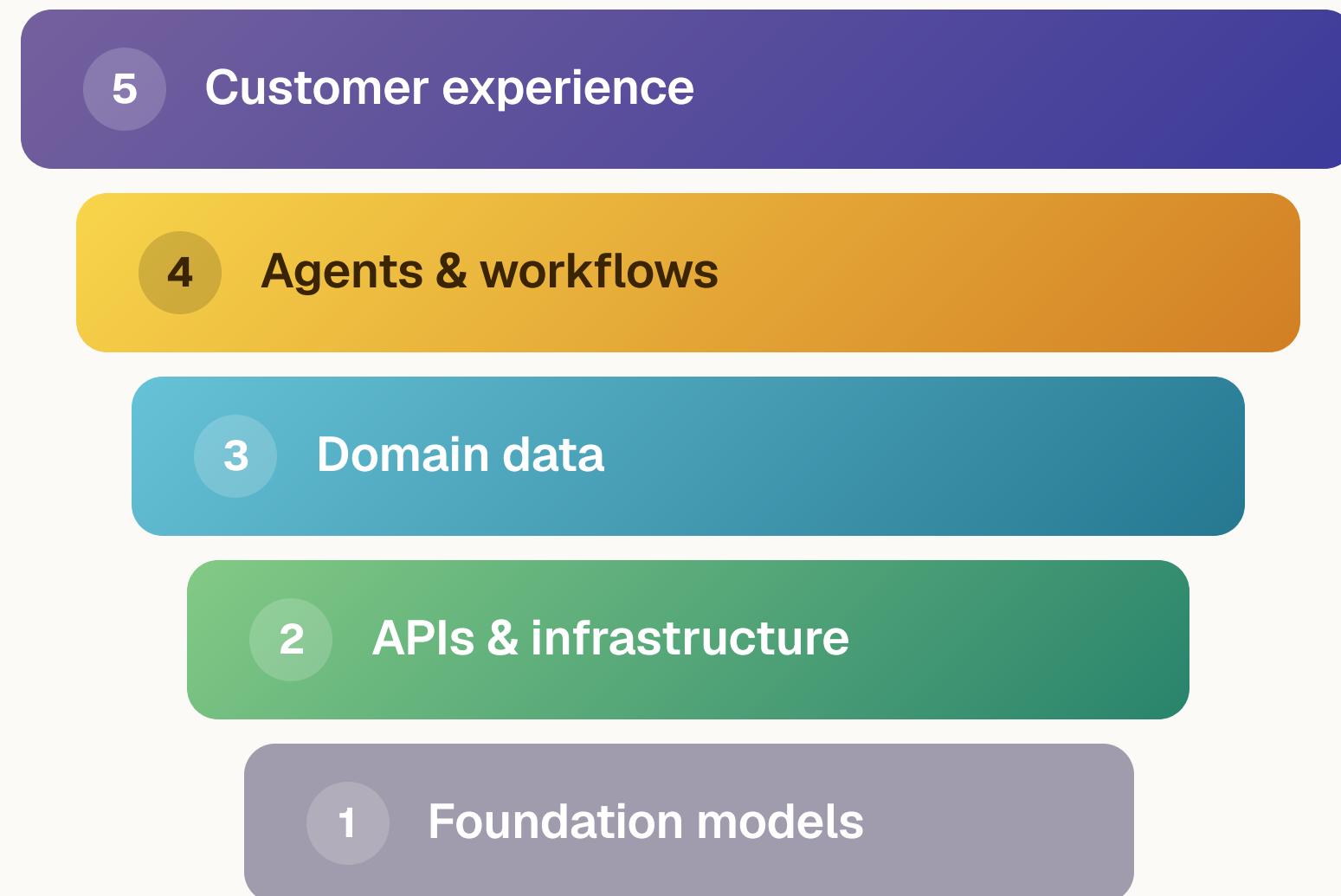
# Beauty is being digitized and AI-ified at the same time.

The AI layer is being built on top of a digital layer that's still wet. *That doesn't happen often.*



# The beauty-tech AI stack

*Five layers · value moves up · build at the top*



VALUE

## LAYER 5

Owner · stylist · end customer · WhatsApp · voice · iPad

## LAYER 4

Discovery · conversion · retention · intelligence

## LAYER 3

Service taxonomy · skin/hair · booking patterns · inventory

## LAYER 2

Google AI · Anthropic · OpenAI · Vapi · Twilio · vector DBs

## LAYER 1

Gemini 3 · Claude 4.6 · GPT-5 · Llama · GPT-OSS

• THE PUNCHLINE

**Layer 1 is becoming free.  
Layers 3 to 5 are where  
defensible  
companies get built.**

*Keep this in mind when you decide what to build this weekend.*





LIVE IN PRODUCTION

# What's actually shipping

*Six categories live today · billions in GMV behind them*

01

## AI receptionists

Voice agents recover 30–40% of missed calls.

02

## Hair colour & makeup try-on

Diffusion inpainting · 2–3× e-commerce lift.

03

## Skin & hair diagnostics

Phone camera → vision model → booking.

04

## Content generation

One owner = a month of Instagram in an afternoon.

05

## Conversational booking

Multi-constraint search a form can't do.

06

## Owner copilots

"Why was last week 12% down?" — with answers.



# ...and a dozen more shipping fast

*Pick one. Build it this weekend.*

---

## Stylist copilot

Mid-shift assistant · client history on tap.

---

## Smart inventory

Auto-reorder when product runs low.

---

## Dynamic pricing

Saturday peak vs Tuesday off-peak.

---

## AI photoshoot

Hair & nail lookbooks · generated, not shot.

---

## Staff training agent

Personalised drills · per-stylist gaps.

---

## WhatsApp catalogue AI

Conversational service & product discovery.

---

## Review reply bot

Tone-matched responses to Google & Insta.

---

## Booking + payments

Pay-to-confirm flow · no-shows drop 40%.

---

## Try-on for nails

AR overlay on phone camera · pre-visit.

*None of these need a new model. They need someone to ship them.*



# Four agents - one business on autopilot

*Lila finds them · Cora books them · Echo brings them back · Sage tells you what to do*

## Lila

### DISCOVERY

**Gets you found online.**

- Google Business Profile, on autopilot
- Local SEO & directory listings
- Review monitoring & replies
- Ranks on ChatGPT & Perplexity

---

Show up first when someone searches the service you offer.

## Cora

### CONVERSION

**Books every inquiry, 24/7.**

- Answers in 30 seconds, every channel
- Phone · WhatsApp · IG · FB · email
- Nurtures, qualifies, then books
- Escalates only when she should

---

Every inquiry becomes a booking. No lead goes cold.

## Echo

### RETENTION

**Brings clients back.**

- Tracks each client's rebooking cycle
- Personal reminders, not blasts
- Flags at-risk clients early
- Win-back for dormant regulars

---

One-time visitors become regulars. Regulars stay regular.

## Sage

### INTELLIGENCE

**Tells you what to do next.**

- Forecasts demand 4+ weeks out
- Optimises staff & inventory
- Plain-English recommendations
- Catches problems before they cost

---

Confident decisions, backed by data. Stop guessing.





WHERE APIS FIT

# Four integration patterns

*Pick yours · A through D, in order of ambition*

A

## Thin wrapper

UI + prompt + LLM call.

*Most hackathon projects.*

B

## RAG · domain-grounded

LLM + your proprietary data.

*The good hackathon projects.*

C

## Tool-using agents

LLM that calls APIs · books · charges · sends.

*The exceptional ones.*

D

## Autonomous agents

Plan → act → observe → re-plan. Runs without you watching.

*The frontier.*



• PATTERN D · EXAMPLE 1 OF 3

# The full-stack receptionist

*A single agent · no human in the loop after pickup*

CONVERSION · VOICE + TEXT

- 01 Answers the call
- 02 Checks 6 stylists' calendars in parallel
- 03 Matches stylist skill ("curly hair colour")
- 04 Pulls last 3 visits from CRM
- 05 Offers 3 slots, books one
- 06 Sends WhatsApp confirmation
- 07 Adds to Google Calendar
- 08 Texts the stylist a heads-up

• LIVE TODAY

DINGG **Cora**, Phonely, Slang.ai, and others ship this in production right now.

## STACK INVOLVED

Vapi or Retell · GPT-5 or Claude · booking API · WhatsApp Business · Google Calendar API

• WHY IT WORKS

Voice agents recover 30–40% of missed calls — the biggest leak in the funnel, now plumbing, not headcount.



• PATTERN D · EXAMPLE 2 OF 3

# The overnight marketing agent

*Runs at 2 AM · sleeps · repeats tomorrow*

- 02:00 AM Agent wakes.
- 02:04 Identifies 47 lapsed customers.
- 02:11 Drafts personalised win-back for each.
- 02:18 Schedules sends 10 AM – noon.
- 02:21 Writes owner a Slack summary, sleeps.

• WHY IT'S HARD

Personalisation needs the customer's full history. Timing matters. Tone matters. *One bad message kills the next six months of trust.*

---

REAL EXAMPLE

DINGG **Echo** · Klaviyo-for-salons is being rebuilt as agents like this.



# The intelligence agent

## Sage

INTELLIGENCE · AD-HOC + DAILY

### OWNER ASKS

*"Why was last week 12% down?"*

### SAGE QUERIES · IN PARALLEL

POS · text-to-SQL   Weather API   Staff schedule   Ad spend

Local events   Past 12 weeks

### DIAGNOSIS

Two top stylists on leave during a rainy weekend that overlapped a competitor's grand opening 600m away.

### • LIVE TODAY

DINGG **Sage** · Hex, Julius, and others doing this horizontally. The vertical version — that knows YOUR business — is the unlock.

### STACK INVOLVED

Text-to-SQL over POS · RAG over ops + reviews · LLM reasoning · Slack / email out

### • WHY IT'S HARD

Horizontal SQL agents hallucinate joins. Vertical ones — that know *this* POS schema and *this* business — don't.



# Why Pattern D is hard

*Four things that separate a demo from production*

## 01 Tool design

Small · well-described · well-scoped functions. Most teams underinvest. Spend time on signatures and docstrings.

## 02 Memory

Across turns, days, customers. Get this wrong → amnesia or hallucination. Decide what persists, what doesn't.

## 03 Observability

Agent did 14 things. Step 11 was wrong. *Which* step? Tracing, logs, replay — boring infra, essential.

## 04 Cost & latency

4 calls × 800 ms × ₹2 × 10,000/day = ₹20K/day. Do the math *before* you demo.





# The LLM *is* the vision model now

*Beauty is visual — and the API for vision just collapsed*

## Gemini 3 Pro and Claude 4.6 are natively multimodal.

Send a photo with your prompt → get structured output. No separate vision API needed for most cases. **10× simpler than 18 months ago.**

### Skin diagnostics

selfie → skin type · concerns · service rec

Gemini 3 multimodal

### Virtual try-on

hair colour · makeup · nails

Replicate · Flux · Imagen

### Before / after analysis

2 photos → marketing copy

Gemini · Claude

### Shelf & inventory recog

identify products from a photo

Gemini · Claude

### Hair texture classify

stylist–customer skill matching

Fine-tuned vision

### In-store crowd density

queue management · staffing

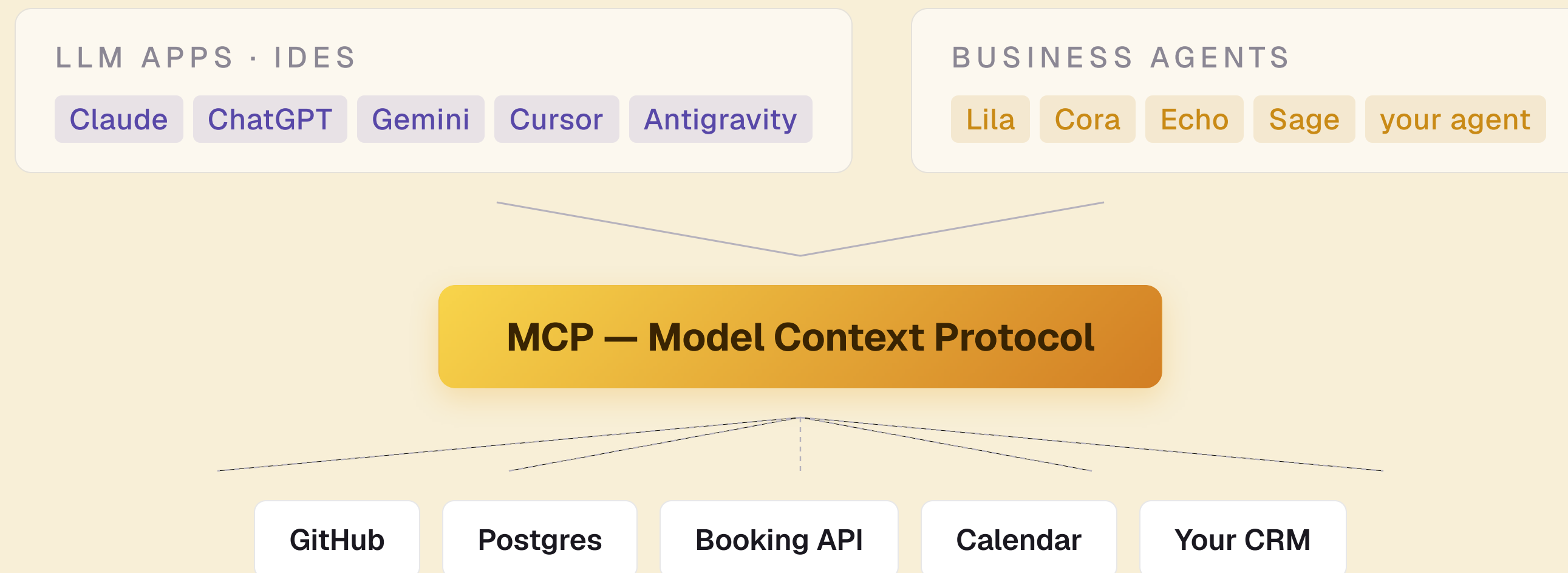
Edge vision · Roboflow



OPEN PROTOCOL

# MCP · the USB-C moment for agents

*One protocol · any agent talks to any tool*



*Customers hook ChatGPT into salon data. Businesses build once, plug everywhere.*

PART 2 · 18 MINUTES

# Agentic Coding

*How to build software with AI agents — properly.*

**Vibe coding is done.** That was 2024–2025 — typing prompts and hoping. What we're doing now is **Agentic Coding** — engineering, with a different interface. Same discipline. Different tools.





# Talk like an engineer, not a prompter

*Four words that change how judges hear you*

## Agent

An LLM that plans, acts, observes, re-plans — in a loop.

→ *Not a chatbot. A chatbot answers. An agent does.*

## Tool call

A function the agent can invoke. The agent's hands.

→ *book\_appointment(stylist, datetime)*

## Context

The agent's working memory for this task.

→ *Prompt + files read + tool outputs + rules file.*

## Spec

Written specification of what you want built.

→ *SPEC.md — most undervalued artifact in the repo.*



# Six more words to know cold

*Use these when you pitch · judges hear an engineer*

## Plan mode

Agent proposes a plan *before* writing code. You approve.

*30 seconds saves 30 minutes.*

## Subagent

A child agent spawned for a sub-task.

*Parallel work · API + UI in tandem.*

## MCP

Open standard for agent ↔ tools. Same protocol everywhere.

*Build once, works with Claude, ChatGPT, Cursor.*

## Rules file

Persistent instructions the agent reads every task.

*CLAUDE.md · AGENTS.md · .cursorrules*

## Artifact

Concrete output the agent hands back.  
Reviewable.

*A diff. A screenshot. A test result.*

## Skill

A reusable, scoped capability the agent invokes on demand.

*"Open browser, run flow, screenshot result."*

- THE SINGLE BIGGEST LEVER

# Write a **SPEC** before you write code.

Brainstorm → Spec → Build.  
Skip this, lose six hours by midnight.

*Your clarity is the bottleneck — not the agent.*



• STEP 1 OF 3

# Brainstorm — *with* the agent

*Open chat, not the IDE. Talk before you type.*

- 01 Open Claude or Gemini in **chat mode**, not the IDE.
- 02 Have a real conversation. Let it interrogate you.
- 03 Answer every question. Push back. Refine.

*30 minutes of talking saves 10 hours of building.*

## PASTE THIS PROMPT

```
"I want to build a salon booking chat  
for a hackathon. Help me think it through.
```

```
Ask me the questions a senior engineer  
would ask before scoping this.
```

```
Don't write code yet."
```



• STEP 2 OF 3

# Generate SPEC.md

*Make the agent write it · then edit it · then commit it*

## PASTE THIS PROMPT

"Now write SPEC.md for this project.  
Cover:

- problem
- users
- scope
- out-of-scope
- data model
- key flows (pseudocode)
- tech stack
- non-goals
- demo script

## What you get back

A 2–3 page document. **Read it. Edit it. Push back on what's wrong.**

## This becomes your source of truth

- Drop it in the repo as SPEC.md
- Reference it from every prompt
- Update it as scope shifts
- Hand it to teammates day-of





• STEP 3 OF 3

# Build from the spec

*Now — and only now — open the agentic IDE*

- 01 Drop SPEC.md in your repo root.
- 02 Tell the agent to read it first.
- 03 Demand a plan before code.
- 04 Approve · execute · verify · ship.

## PASTE THIS PROMPT

```
"Read SPEC.md.
```

```
Propose an implementation plan  
for Phase 1 only.
```

```
Show me the plan before  
writing any code."
```



# System design still matters

*More than ever — the agent won't figure out architecture*

## Boundaries

What's a service? A module? A function? Agents amplify whatever structure exists. Good boundaries → clean diffs.

## Data flow

Who writes? Who reads? Where's the source of truth? Get this wrong and the agent will spread bugs everywhere.

## State

Session, persistent, derived — three different storage strategies. Decide which lives where, on day one.

## Failure modes

What happens when the LLM is slow, wrong, or down? Build for the unhappy path. Always.

*Spend 20 minutes drawing a box-and-arrow diagram on paper. Photograph it. Hand it to the agent — now it understands what you understand.*



## MENTAL MODEL

# Treat the agent like a junior engineer

*...with super-speed · the single best framing*

*A junior engineer...*

Does what you ask · misses what you meant

Won't push back on bad ideas — *you* have to

Produces plausible code that doesn't work

Gets better with more context

Needs review before shipping

*The agent...*

***Same.***

*Just faster.*

*You wouldn't let a junior push straight to production. Don't let the agent either.*





APPLY THE RIGOR

# The five disciplines

*Apply the same engineering rigor you'd apply to a junior*

01

## Give clear constraints

"Use React not Vue. Postgres not Mongo. Don't touch auth.ts."

02

## Demand a plan before code

Always. Even for "small" changes that aren't small.

03

## Enforce tests

Non-negotiable. After every change.

04

## Gate changes — small diffs

Big rewrites collapse. Small diffs win. Reject more than you accept.

05

## Assume outputs can be wrong

Especially auth, payments, edge cases, security. Read every line.

• TESTING

# How you make agents **actually** useful.

AI-generated code is statistically dirty. Subtle conceptual bugs that pass type checks.

The only defense: **executable verification.**



IN ORDER OF IMPORTANCE

# Three testing layers

*First two are free · the third is on you*

---

01

## The agent runs its own tests

Add to your rules file: "After every change, run the test suite. If it fails, fix it. Loop until green." Catches ~70% of mistakes for free.

---

02

## The agent uses the browser to verify itself

Antigravity has built-in Chromium. Claude Code can screenshot. Tell the agent to open the page, run the user flow, screenshot the result. Visual verification catches what unit tests miss.

---

03

## YOU demo it before submission

End-to-end. Twice. If it breaks at 11 PM, you have time. If it breaks at 5 minutes to submission, you don't.

---



COPY-PASTE THIS

# Add this to your rules file

*One paragraph · turns "code that compiles" into "code that works"*

COPY-PASTE INTO CLAUDE.MD / AGENTS.MD / ANTIGRAVITY RULES

Before marking any task complete:

- (1) run the test suite
- (2) open the affected page in the browser
- (3) execute the user flow described in SPEC.md
- (4) screenshot the result

*This single paragraph is the difference between agentic vibing and agentic engineering.*



# Frontend

*What ships demos in 24 hours*

• RECOMMENDED

## Next.js

Best agent support · full-stack in one repo  
· Vercel deploys in 30 seconds. **Default for anything web.**

Second choice

## Plain React (Vite)

Faster to start, but needs a separate backend → friction. Pick this when you already have an API.

• WILDCARD

## Vanilla HTML + Tailwind

Single file · zero build · agent ships instantly. Use when the "app" is really a polished mockup.

*Boring stacks ship demos. Exotic stacks lose hackathons.*





# Backend

*Python or Node? Depends on what you're building*

• BEST FOR AI-HEAVY

## Python · FastAPI

- All LLM SDKs are Python-first
- Vector DBs · ML libs · data pipelines
- Agent writes idiomatic FastAPI easily
- Deploy on Railway · Render · Modal

• BEST FOR FULL-STACK ONE-LANG

## Node · TypeScript

- One language: frontend + backend
- Next.js API routes · no separate service
- Covers 80% of hackathon backend needs
- TypeScript catches bugs the agent misses

**Rule of thumb:** *AI-heavy* → *Python* · *CRUD with chat UI* → *Next.js end-to-end*.



# Database

*Supabase · Postgres · SQLite · Mongo*

## Supabase

• BEST DEFAULT

Postgres + auth + storage + realtime + row-level security — all in one. 10 minutes to a working DB.

## Postgres raw

*Neon · Railway · When you DIY auth*

Leaner if you don't need Supabase's auth. Cheaper. Just as fast.

## SQLite

*Underrated for solo demos*

Single file · zero setup · fast · ship the .db with your repo. Limit: one machine.

## MongoDB

*Don't.*

Fine for unstructured data — but bookings, services, customers, staff *are* relational. You'll want joins by hour 8.

*Beauty-tech specific: services · bookings · customers · staff = all relational. Use Postgres.*



# Deployment

*Where speed matters · pick the boring one*

## Vercel

Next.js → git push → live in 30 seconds.

→ *Frontend / full-stack JS*

## Railway · Render

Python + Postgres · one command.

→ *FastAPI backends*

## Modal · Fly

Python with GPUs · background jobs.

→ *Advanced async work*

## Cloudflare Workers + D1

Serverless · edge · cheap.

→ *If you know what you're doing*



- THE 2026 DEFAULT STACK

Next.js + Supabase + Vercel  
+ Gemini / Claude API + Antigravity or Claude Code

Set up in 30 minutes · builds anything · deploys in seconds.

*Pick this. Spend zero hours debating it. Build for 24.*



SAME LOOP · 5 MIN OR 5 HOURS

# The agentic coding loop

*Write this on your monitor for 24 hours*

- 
- |    |                 |                                                                 |
|----|-----------------|-----------------------------------------------------------------|
| 01 | <b>Research</b> | "Explore the repo. Tell me what's here. Don't change anything." |
|----|-----------------|-----------------------------------------------------------------|
- 
- |    |             |                                                |
|----|-------------|------------------------------------------------|
| 02 | <b>Plan</b> | "Propose a plan. Show me before writing code." |
|----|-------------|------------------------------------------------|
- 
- |    |                |                                               |
|----|----------------|-----------------------------------------------|
| 03 | <b>Execute</b> | "Implement the approved plan. Small commits." |
|----|----------------|-----------------------------------------------|
- 
- |    |               |                                                            |
|----|---------------|------------------------------------------------------------|
| 04 | <b>Review</b> | "Run tests. Verify in the browser. Screenshot the result." |
|----|---------------|------------------------------------------------------------|
- 
- |    |             |                                               |
|----|-------------|-----------------------------------------------|
| 05 | <b>Ship</b> | "Write README. Update SPEC.md. Commit. Push." |
|----|-------------|-----------------------------------------------|
-



SAVE THIS SLIDE · PHOTOGRAPH IT

# The hackathon cheat code

*10 tips · in priority order*

- 01 Spec before code. 30 min brainstorm → SPEC.md → build.
- 02 Draw architecture on paper. Hand the photo to the agent.
- 03 Write a rules file in the first 30 min. Compounds for 24 hours.
- 04 Plan mode for anything non-trivial. 30 sec saves 30 min.
- 05 Tests + browser verification on every change. Non-negotiable.
- 06 Small diffs. Review every one. Reject more than you accept.
- 07 Boring stack — Next + Supabase + Vercel, or FastAPI + Postgres + Railway.
- 08 Pick ONE agent IDE. Don't tool-shop at 2 AM.
- 09 Save the last 30 min for README + demo script. Judges read READMEs.
- 10 When you pitch — name your moat. What does your software know that ChatGPT doesn't?

PART 3 · 10 MINUTES

# Where this is going

*Some predictions will be wrong. Most will be early. The direction is real.*



# 10 predictions for beauty tech

*First five*

01

## The booking form dies

Replaced by a conversational front door. WhatsApp, voice, in-app — pick.

02

## 24/7 invisible receptionist

Default infrastructure for any salon over four chairs.

03

## Try-before-you-sit

Hair colour, makeup, nail try-on becomes table stakes on every booking flow.

04

## Skin & hair, measured not guessed

Phone camera → diagnosis → personalised service → booking, in one flow.

05

## Every stylist gets a copilot

Mid-shift client history on tap. Personal service at scale, finally.





# 10 predictions for beauty tech

*Last five*

06

## Content collapses to one person

Owner makes a month of Instagram in an afternoon. SMB agencies extinct.

07

## Vertical AI eats horizontal

A salon-specific agent beats ChatGPT by 35 points. Repeats across every SMB vertical.

08

## Salon org chart shifts

Owner becomes a conductor. 4–6 agents inside the business. Same revenue, half the back-office.

09

## Dynamic personal pricing

Yield management arrives in beauty. Tuesday 11 AM costs less than Saturday 4 PM — and the customer is offered the trade.

10

## Portable beauty profile

Your hair history travels with you. Whoever builds this layer owns a category-defining position.



APPLY TO EVERY IDEA

# Three lenses

*Yours · ours · anyone's*

## 01

### Where on the stack?

Layer 1–2 commodity. Layer 3–5 value.

---

*What layer are you really building at?*

## 02

### Horizontal or vertical?

Vertical wins in SMB. Always.

---

*What does your software know that ChatGPT can't?*

## 03

### Augment or replace?

Best products augment humans. Worst try to replace and fail at the last 5%.

---

*Who's the human you're making 10× better?*



# Now — go build.

Beauty is one of the last big industries to digitize.  
You're sitting in front of agent tools that didn't exist 12  
months ago.

*Build vertically.*

*Manage agents like a team. Ship.*

---

**Santosh Patidar** · Co-founder, DINGG AI · [dingg.ai](https://dingg.ai)



• CONNECT WITH ME

**Scan to find me on LinkedIn**

*Questions? DMs are open.*